CogniMem: A Cognitive Hybrid Memory Architecture for Long-Term Personalized LLM Agents Using Hybrid RAG, Temporal Knowledge Graphs, and Learned-Plus-Rule Memory Scoring

Joseph Remingston L
Reg. No. 23BCE8134
B.Tech CSE (Artificial Intelligence and Machine Learning)
School of Computer Science and Engineering, VIT-AP University
Amaravati, Andhra Pradesh, India

Sriramaneni Suhas
Reg. No. 23BCE7367
B.Tech CSE (Artificial Intelligence and Machine Learning)
School of Computer Science and Engineering, VIT-AP University
Amaravati, Andhra Pradesh, India

Kumar Shreenagesh
Reg. No. 23BCE9201
B.Tech CSE (Artificial Intelligence and Machine Learning)
School of Computer Science and Engineering, VIT-AP University
Amaravati, Andhra Pradesh, India

Tejesh Madanwad
Reg. No. 23BCE7365
B.Tech CSE (Artificial Intelligence and Machine Learning)
School of Computer Science and Engineering, VIT-AP University
Amaravati, Andhra Pradesh, India

***Abstract*—Conversational large language models (LLMs) forget: context windows are finite, and storing every utterance verbatim and retrieving it by embedding similarity ignores what a personal assistant actually needs—decisions about what to keep, awareness that facts change, and evidence for what it says. A recent IEEE Access literature review surveyed vector, graph, and state-machine memories and argued for hybrid designs, but did not implement one at scale. This paper presents CogniMem, a local-first memory system that turns that argument into working software. CogniMem redacts sensitive patterns, classifies statements into six memory categories, scores importance with a learned-plus-rule hybrid (25% trained XGBoost/neural ensemble, 75% rules), assigns working, short-term, long-term, and archive tiers, resolves task and fact conflicts while preserving revision history, and stores relations in a bitemporal knowledge graph. At query time it fuses keyword, vector, and graph channels by weighted reciprocal-rank fusion, re-ranks with a local cross-encoder, applies a time- and link-aware structural reorder, and returns cited answers with optional abstention. A four-arm ablation on a controlled 360-question benchmark, together with LoCoMo (1,973 questions), LongMemEval, and a synthetic importance dataset, shows that adding the graph raises Recall@5 from 89.7% (vector only) to 94.1%, and the full pipeline reaches 97.5% (from 93.0% before an audit-driven redesign); LoCoMo Recall@5 rises from 55.8% to 66.8% and fact retention from 21.0% to 90.6%. We also report a negative result: the trained importance model reaches macro-F1 0.891 on its synthetic test split yet changes no measured retention outcome and slows replay 41-fold. The code and evaluation protocol are publicly available.**

***Index Terms*—conversational memory, retrieval-augmented generation, temporal knowledge graph, hybrid retrieval, importance scoring, memory consolidation, long-term personalization, LLM agents**

I. Introduction

Large language models (LLMs) power assistants that are expected to know their users, yet each model call is stateless and the context window is finite. Giving an assistant a durable memory therefore means deciding what to store, how to keep it consistent as the user's life changes, and how to retrieve it as evidence at answer time. The common baseline—embed every message and fetch the nearest neighbors for each query, as in retrieval-augmented generation (RAG) [2]—breaks down in predictable ways. Greetings and obsolete reminders compete with useful facts; repeated statements waste space; a newer fact ("I moved to Bengaluru") contradicts an older one ("I live in Chennai"); and semantic similarity is blind to time and relations. The question "Where did I live before Bengaluru?" needs history, "Where is the company I work for based?" needs a two-edge evidence path, and any personalized answer should show which memories support it.

A recent IEEE Access literature review by Akbar *et al.* [1] synthesized 113 papers selected with the PRISMA methodology. It concluded that vector databases alone cannot supply the semantic, episodic, procedural, and emotional memory that agentic systems need, and that hybrid architectures combining vector search, knowledge graphs, and finite state machines are the promising direction. The review is, by design, a survey: its proof of concept used TF-IDF vectors over 12 documents and 20 queries. Several elements that a deployed memory needs are therefore left open—an intelligent memory lifecycle (importance scoring, consolidation, forgetting), temporal reasoning, conflict resolution for contradictory memories, automatic memory classification, adaptive multi-signal retrieval, a learned model of what to remember, and an experimental comparison against modern memory systems.

This paper presents CogniMem, a local-first memory system that turns the review's recommendation into working, measured software. Our guiding research question is: *how much does temporal and relational graph retrieval improve vector-based conversational memory retrieval and RAG?* The contributions are:

1) **A layered memory architecture** that redacts, classifies, scores (with a learned-plus-rule importance score), tiers, reconciles, and persists conversational statements, with an

append-only revision log as the source of truth and rebuildable vector and graph indexes.

2) **A bitemporal knowledge graph and conflict handling** that keep superseded facts with their valid and recorded times, supporting "as-of" and "known-at" queries and multi-hop evidence paths.
3) **A hybrid retrieval pipeline**—weighted reciprocal-rank fusion over keyword, vector, and graph channels, a wide candidate pool, local cross-encoder re-ranking, and a time- and link-aware structural reorder—followed by cited, optionally abstaining answers.
4) **A controlled evaluation** with a four-arm ablation that answers the research question, external benchmarks (LoCoMo, LongMemEval), paired bootstrap confidence intervals, an explicit measurement audit, and a reported negative result for the trained importance model.
5) **An open, deployable implementation** with a Python API, CLI, and an authenticated FastAPI service with per-user isolation, export, and deletion.

The rest of the paper is organized as follows. Section II reviews related work, Section III describes the system, Section IV the experimental setup, Section V the results, Section VI discusses findings and limitations, and Section VII concludes.

II. Related Work

A. Memory for Conversational LLM Agents

Surveys such as [1] catalogue the memory types an agent may need—semantic, episodic, procedural, and emotional—and the stores used to hold them. Among systems, MemGPT [3] manages a tiered virtual context that the LLM itself pages in and out, drawing an analogy to operating-system memory hierarchies. Mem0 [4] extracts salient facts from conversations and maintains them through add and update operations, with a graph-based variant that records relations. MemoryBank [5] updates memories with a mechanism inspired by the forgetting curve. These systems typically rely on an LLM to decide what to write or revise. CogniMem takes a complementary, inspectable route: retention, conflict, and task decisions are deterministic rules blended with a small trained scorer, so every write can be traced and replayed from a revision log, and the intake path runs locally. We do not claim superiority over these systems; a head-to-head comparison is future work.

B. Vector Retrieval, Fusion, and Re-ranking

Dense retrieval over embeddings, with approximate nearest-neighbor indexes such as those in FAISS [6], underlies most RAG pipelines [2]. CogniMem embeds text with a local BGE encoder [7]. Because different channels produce incomparable scores, we merge ranked lists with reciprocal-rank fusion [8] and then re-rank candidates with a cross-encoder [9] built on the MiniLM architecture [10]. Cross-encoders read the query and passage jointly and are typically more accurate, but costlier, than similarity between independently computed embeddings.

C. Knowledge Graphs and Time

Knowledge graphs encode entities and typed relations explicitly [11], enabling multi-hop evidence paths that similarity search overlooks; [1] also notes that graph-based systems struggle with temporal data. The database literature distinguishes valid time (when a fact is true in the world) from transaction time (when the database learned it) [12]. CogniMem adopts this bitemporal view for conversational facts, which makes it possible to answer both "what was true then?" and "what did the system know then?".

D. Benchmarks

LoCoMo [13] provides very long multi-session dialogues with questions grounded in annotated evidence turns, and LongMemEval [14] tests the long-term interactive memory of chat assistants. Section IV describes how each is used, together with the synthetic importance dataset on which CogniMem's learned scorer is trained.

E. Gap Analysis

Table I maps the limitations of [1] that motivate this work to the CogniMem components that address them.

TABLE I
Limitations of the Base Review and the CogniMem Response

Limitation of [1]	CogniMem component
No implemented hybrid; reliance on vectors	Keyword + vector + graph channels fused by RRF
No memory lifecycle	Importance score, four tiers, expiry, consolidation, cleanup
Weak temporal reasoning	Bitemporal graph, as-of / known-at queries, structural reorder
No conflict resolution	Fact and preference supersession with revision history
No automatic classification	Six-category rule-based classifier
No learned retention model	Importance = 25% XGBoost + neural ensemble, 75% rules
No evaluation against modern settings	Four-arm ablation, LoCoMo, LongMemEval, bootstrap CIs

III. System Design

CogniMem is organized as six layers: understanding and structuring input, scoring and change management, persistence, retrieval, answer construction, and interfaces. The first three form the write path (Fig. 1) and the next two the read path (Fig. 2). Four principles run through all layers: (i) *local first*—intake, embedding, graph search, and re-ranking run on the user's machine; (ii) *inspectable*—an append-only JSONL revision log is the source of truth and all indexes can be rebuilt from it; (iii) *conservative*—corrections add revisions rather than overwrite, and ambiguous task references are recorded rather than guessed; and (iv) *evidence-bound*—answers must cite the memories that support them or abstain.

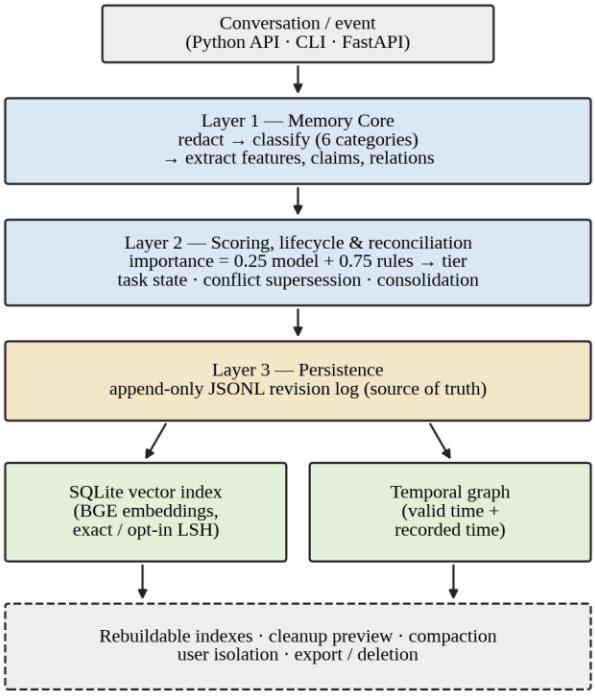


Fig. 1. Write path of CogniMem: intake and classification, scoring with lifecycle and reconciliation, and persistence with rebuildable indexes.

A. Intake and Classification (Layer 1)

A *MemoryCore* receives a statement with user and session identifiers, a role, a timestamp, and optional metadata. Supported sensitive patterns are redacted before scoring or storage. The statement is classified into one of six categories (Table II) and, optionally, split into separate records when it contains clearly independent clauses. The core extracts numeric cues (deadlines, preference and task language, entity density, recency, interaction signals, category), turns supported factual statements into claims, and accepts explicit typed entities and relations from callers for graph indexing. Natural-language relation extraction covers supported patterns rather than arbitrary prose.

TABLE II
MEMORY CATEGORIES

Category	Example	Typical tier
Semantic	"My supervisor is Priya."	Long-term
Episodic	"I submitted the proposal yesterday."	Score-dependent
Procedural	"First run tests, then deploy."	Score-dependent
Preference	"I prefer brief explanations."	Long-term
Task	"Remind me to submit the report Friday."	Short-term
Temporary	"Thanks!"	Working

B. Importance Scoring and Memory Tiers (Layer 2)

The default *ImportanceScorer* is a hybrid: a quarter of the score comes from a trained model and three quarters from deterministic rules,

$$s = 0.25\, s_{model} + 0.75\, s_{rule} \quad (1)$$

where the rule score combines the extracted features and s_{model} is the value of the class predicted by the model—low, mid, or high, with class values 0.3102, 0.5792, and 0.8372 (the mean training importance of each class). As the rules do, the model part is multiplied by 0.25 for small talk, because the model never saw conversation in training and otherwise rates "hello" as highly important.

The model is an ensemble trained on *csv5k*, 5,000 AI-generated memory sentences (443 distinct) with synthetic importance scores from 0.20 to 1.00, cut into low (below 0.4), mid, and high (0.7 and above); every copy of a sentence stays in one split (3,200 / 792 / 1,008 rows). The XGBoost member [15] uses 525 features (13 text features plus 512 hashed word counts; depth 3, minimum child weight 3, 304 rounds). The neural member freezes a local 384-dimensional encoder (BAAI/bge-small-en-v1.5) and combines it with the same 13 features in a two-branch head (text 384→192→128, features 13→64→32, joint 160→128→64→3), averaging three seeds. Both are temperature-calibrated and blended,

$$p = 0.5\, p_{xgb} + 0.5\, p_{nn} \quad (2)$$

and class weights of 1, 0.7, and 1 pick low, mid, or high, with all choices made on validation data only. Setting the scorer to rules-only is a single switch, and each stored record notes which scorer ran together with the class, probabilities, and both part-scores.

A *LifecycleManager* maps the score and category to one of four tiers. Working memory (temporary items or scores below 0.25) expires after one hour; short-term memory lasts 14 days, or 90 days for what users say about their own lives; long-term memory is reserved for durable categories scoring at least 0.42 and is shown as archive after 90 inactive days; and dated tasks expire 24 hours after their deadline. These are policy decisions, not human-labeled truths.

C. Tasks, Conflicts, and Consolidation

Tasks carry due dates and support completion, cancellation, rescheduling, and recurrence. Task state is stored separately from lifecycle tier; normal retrieval hides expired and resolved items, while history controls expose them. When a supported fact or preference conflicts with an earlier one, CogniMem marks the newer assertion as current and keeps the older record and its revisions, so "I live in Chennai" followed by "I moved to Bengaluru" can answer both current and historical questions. Repeated equivalent memories can be consolidated into a canonical summary listing the supporting memory IDs, sessions, and observation dates. Cleanup is previewed by default; the balanced policy protects expired memories scoring at least 0.65, durable categories, legal holds, referenced evidence, and unresolved tasks, and applying it removes only eligible expired data, compacts exact duplicate revisions, and rebuilds the indexes.

D. Persistence and Temporal Graph (Layer 3)

Serialized *MemoryRecord* revisions are appended to a JSONL file under cross-process locks; a record holds its category, tier, score, timestamps, task state, claim and conflict status, evidence IDs, and source metadata. SQLite stores chunks and embeddings, graph nodes, edges and aliases, and index state. Vector search is an exact scan by default, with an

opt-in locality-sensitive-hashing (LSH) path for larger collections.

The graph stores typed relations such as *lives_in*, *works_at*, *based_in*, *likes*, and *has_task*, each linked to its source memory. Each relation carries a valid-time interval $[t_s, t_e)$ and a recorded time t_r. A relation e is visible to a query with valid-time point τ_v (as-of) and knowledge-time point τ_k (known-at) when

$$visible(e) \Leftrightarrow t_s \leq \tau_v < t_e \wedge t_r \leq \tau_k \quad (3)$$

Multi-hop retrieval returns the evidence path, and graph connectivity alone is never treated as proof of a fact that was not stated.

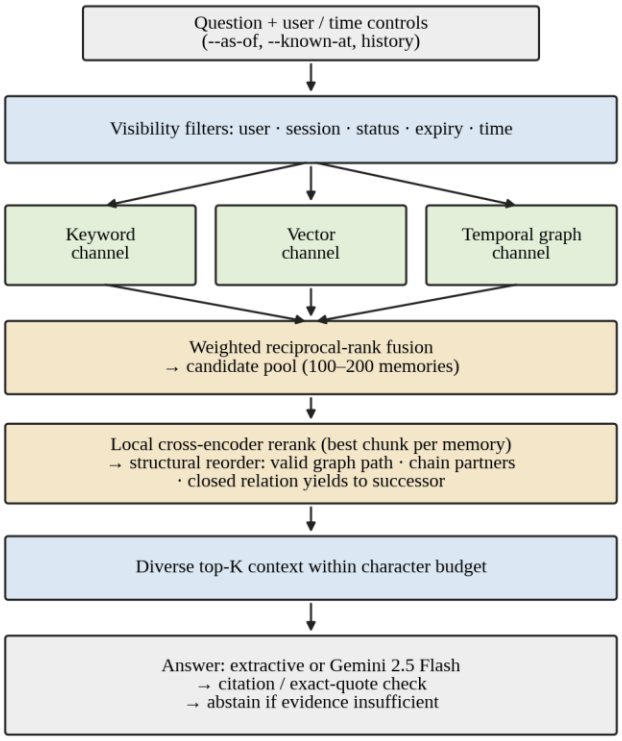


Fig. 2. Read path of CogniMem: filtered multi-channel candidate generation, rank fusion, cross-encoder re-ranking with structural reorder, and cited answers.

E. Hybrid Retrieval and Re-ranking (Layer 4)

After filtering by user, session, history, task state, expiry, and time, the retriever gathers candidates from three independent channels: keyword, vector, and graph. Ranked lists are merged by weighted reciprocal-rank fusion (RRF) [8],

$$s(m) = \Sigma_{c \in C} w_c / (k + r_c(m)) \quad (4)$$

where $r_c(m)$ is the rank of memory m in channel c, w_c the channel weight, and k a smoothing constant. The fused list is cut to a candidate pool of min(200, max(100, 4K)) memories for top-K retrieval; fusion order is computed at the old depth and deeper candidates are appended, so a wider pool never reorders results when re-ranking is off or fails. Up to three representative chunks per candidate are scored by a local cross-encoder (ms-marco-MiniLM-L-6-v2); each memory keeps its best chunk, and the fusion score and ID break ties. If the model fails, fused results are returned with fallback metadata.

A structural step then reorders the reranked list using time and graph links. Candidates on a cued, currently valid graph path move up and the other memories on that path follow; and when the question asks about a relation that has since closed, the closed statement yields to its successor. This targets a measured weakness of the cross-encoder, which scores textual relevance but cannot see validity intervals or links. For comparison, the lightweight keyword-only search in the store ranks by

$$s(m|q) = 0.65\, s_{kw} + 0.10\, s_{cat} + 0.05\, s_{tier} + 0.10\, s_{rec} + 0.10\, s_{imp} \quad (5)$$

(keyword relevance, category, tier, recency, importance) and requires lexical overlap with the query.

F. Context, Answers, and Abstention (Layer 5)

The context selector keeps the reranked order, drops near-duplicate snippets, and fits a character budget; graph paths are included only when all supporting memories fit. The local extractive generator returns deterministic evidence text, and an opt-in flag makes it refuse typed questions (name, place, occupation, amount, duration) that no confidently reranked memory answers; every generator answers "insufficient evidence" when nothing is retrieved. A hosted generator (LangChain with Gemini 2.5 Flash [16]) must return source IDs and exact supporting quotes found in the supplied context; answers with invalid citations are rejected. Quote checking verifies provenance but cannot by itself prove every semantic inference. Memory text leaves the machine only when a hosted answer is requested.

G. Interfaces and Operations (Layer 6)

A Python API, a command-line interface, and a FastAPI service expose the system. The service adds signup and login, bearer-token authentication, role checks, per-user memory endpoints, search, JSON/CSV export, account deletion, liveness and readiness endpoints (readiness reports the active importance scorer), structured request logs, and Prometheus metrics; user identity is derived from the authenticated token. Settings are validated at start-up and the importance model and encoder are loaded there, so a misconfiguration stops the service instead of failing individual requests.

IV. Experimental Setup

A. Datasets

Table III summarizes the evaluation suite. The *controlled benchmark* comprises twelve personas, 456 memories, and 360 questions (12 query types × 30: semantic, episodic, temporal, historical, multi-hop, entity-relationship, conflict, preference, procedural, recency, negative, and unanswerable). It was authored by an AI assistant and is *not* human-validated. All memories belong to one user; the persona is the speaker and every question names the persona, so other personas' memories act as distractors. Personas p01–p03 form a dev split (90 questions) used for tuning; p04–p12 form the test split (270 questions: 247 answerable, 23 unanswerable) used for every reported number. Validators check that each answer appears in a supporting memory, each historical question has a relation valid at its as-of date, and each multi-hop pair shares an entity.

TABLE III
EVALUATION SUITE

Dataset	Size	Used for
Controlled benchmark	12 personas, 456 memories, 360 Q	Retrieval, answers
LoCoMo [13]	10 conversations, 1,973 Q	Evidence retrieval
LongMemEval [14] (oracle)	457 examples, 9,831 turns	Retention
csv5k (synthetic)	5,000 rows, 86 test sentences	Importance model
Classification cases	60 cases	Category / tier
Lifecycle checks	16 checks	Tasks, supersession

On LoCoMo we measure evidence retrieval only, over all ten generated conversations; conversations 1–3 were used to choose the reranker pool size and the other seven are reported as held-out. LongMemEval oracle sessions (436 of 457 examples have labelled answer turns) test whether answer turns survive the retention pipeline; because oracle sessions remove distractors, they measure retention, not question answering. csv5k sentences are short (mean 6.2 words, at most 13) and carry synthetic importance scores with mean 0.708; 86 of the 410 repeated sentences have disagreeing labels, which caps achievable accuracy. Splits are grouped by sentence (stratified 5-fold, seed 42), giving 286 / 71 / 86 train / validation / test sentences; the test split has 1,008 rows (120 low, 272 mid, 616 high) but only 86 distinct sentences, so the sentence-level bootstrap interval is the honest one. The 60 classification cases are AI-authored and were labelled before the classifier ran.

B. Configurations

The research question calls for an ablation that holds data, prompt, answer policy, context size, and procedure fixed while varying the retrieval arm: vector-only, vector + reranker, vector + graph, and vector + graph + reranker; the shipped configuration is the hybrid (keyword + vector + graph) with cross-encoder and structural reorder. We report three snapshots: *baseline* (12 October 2026, the rebuilt harness before any fix: a 20-candidate reranker pool and approximate search on by default), *final* (12 October 2026, all fixes with the rules-only importance score), and *current* (16 October 2026, the shipped default with the hybrid importance score). Only retention, classification, and lifecycle measurements use the importance score; the benchmark and LoCoMo runners never call it, so for their metrics current equals final. Answers come from the local extractive generator, so answer metrics measure grounding and abstention rather than fluency; the hosted Gemini arm is not evaluated here.

C. Metrics

Retrieval is scored over answerable questions with Recall@K, Hit@K, MRR, and nDCG@10, where

$$Recall@K = |G \cap R_K| / |G| \quad (6)$$

for gold evidence set G and top-K list R_K. *Pool hit* records whether gold evidence reached the reranker's candidate pool, and *correct first* is the share of questions whose top-ranked memory is a supporting one. *Grounded correctness* requires that the answer did not abstain and that an accepted answer form appears in a supporting memory present in the context; a different persona's snippet with the same word does not count. For abstention we report the share of answerable questions still answered, the share of unanswerable questions refused, and their balanced mean. Differences between configurations use paired bootstrap 95% confidence intervals [17] and sign-flip randomization p-values. Latency is the median (P50) search time.

D. Implementation

CogniMem is written in Python 3.13. The BGE encoder and the MiniLM cross-encoder run locally and are downloaded on first use; indexes use SQLite. The importance-model files (2.5 MB, checksummed) ship with the code and can be rebuilt bit for bit in about 30 s. All measurements were taken on a single development laptop; a full measurement run takes about an hour.

V. RESULTS

A. Overall Effect

Table IV and Fig. 3 compare the baseline and final snapshots. On the benchmark test split, Recall@5 rises from 93.0% to 97.5% (+4.5 points, 95% CI [2.6, 6.5], p = 0.0002), and the correct memory is ranked first for 84.2% of answerable questions, up from 77.3% (+6.9, [3.6, 10.5]). On LoCoMo, Recall@5 rises from 55.8% to 66.8% (+11.0, [9.5, 12.5]). Fact retention on LongMemEval increases from 21.0% to 90.6%. Grounded correctness on answerable questions improves from 97.6% to 99.2%, but only four of 247 questions changed (all became correct), so the difference is not significant (p = 0.13). Because the benchmark and LoCoMo never call the importance score, the current snapshot is identical to final on these rows.

TABLE IV
BASELINE VERSUS FINAL (TEST SPLIT)

Metric	Base	Final	Change [95% CI]
Bench. Recall@5	93.0	97.5	+4.5 [2.6, 6.5]
Bench. correct first	77.3	84.2	+6.9 [3.6, 10.5]
Bench. MRR (×100)	90.4	93.9	+3.5 [1.6, 5.6]
Bench. nDCG@10 (×100)	87.9	93.3	—
Bench. multi-hop R@5	68.2	100.0	—
Bench. grounded correct	97.6	99.2	+1.6 [0.4, 3.2], n.s.
LoCoMo Recall@5	55.8	66.8	+11.0 [9.5, 12.5]
LoCoMo Hit@5	61.1	72.3	+11.3 [9.6, 12.9]
LoCoMo MRR@5 (×100)	50.5	58.9	—
LoCoMo Recall@10	60.0	73.4	—
LongMemEval retention	21.0	90.6	+69.6 pts
P50 latency, bench. (ms)	85	246	≈3× slower
P50 latency, LoCoMo (ms)	119	435	≈3.7× slower

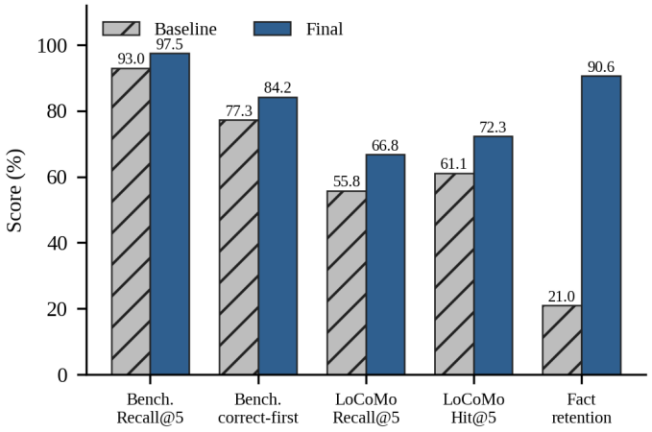


Fig. 3. Baseline versus final on the benchmark test split, LoCoMo, and LongMemEval fact retention (all in %).

B. Ablation: What Does the Graph Add?

Table V and Fig. 4 answer the research question on the benchmark test split. Vector search alone reaches 89.7% Recall@5. Adding the graph channel lifts it to 94.1% (+4.4 points), and adding the reranker to vector alone gives 93.7%. The full vector + graph + reranker arm reaches 97.5%, so the graph contributes 3.8 points on top of the reranker and the reranker 3.4 points on top of the graph. The baseline row is instructive: before the structural reorder, the reranker *hurt* the graph arm (94.1% to 92.4%), because it scores text relevance and discards the time and link evidence that the graph supplies. The structural step repairs this, and the shipped hybrid (which also adds the keyword channel) matches it at 97.5%.

TABLE V
ABLATION ARMS: BENCHMARK RECALL@5 (TEST SPLIT)

Arm	Baseline	Final
Vector	89.7	89.7
Vector + reranker	92.4	93.7
Vector + graph	94.1	94.1
Vector + graph + reranker	92.4	97.5
Hybrid + reranker (shipped)	93.0	97.5

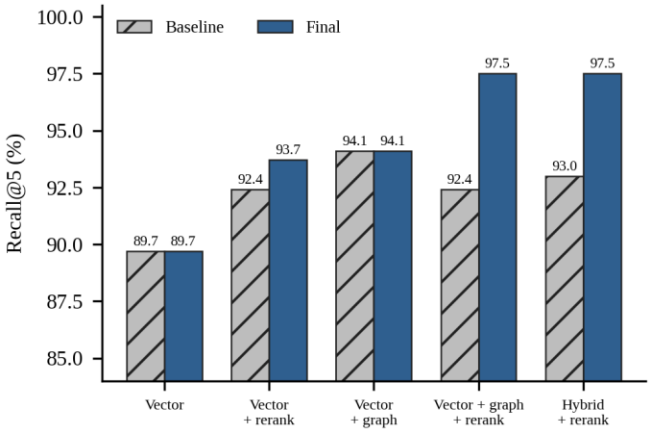


Fig. 4. Benchmark Recall@5 for each ablation arm, baseline versus final (%). The y-axis starts at 84%.

C. External Evidence Retrieval

Table VI isolates the effect of re-ranking on LoCoMo. Cross-encoder re-ranking of the fused pool raises Recall@5 from 48.2% to 66.8% and MRR@5 from 0.374 to 0.589, while median latency rises from 22 ms to 435 ms. Gold evidence reached the candidate pool for 91.9% of questions, and the seven conversations not used to choose the pool size give the same Recall@5 (66.8%), indicating that the pool choice did not overfit conversations 1–3. Opt-in approximate (LSH) search is within 0.3 points of exact search at the larger pool. The graph channel alone finds almost no LoCoMo evidence (Recall@5 of 1.2%) because raw dialogue turns carry no explicit relations; graph benefit therefore depends on relations being extracted or supplied.

TABLE VI
LOCOMO EVIDENCE RETRIEVAL (1,973 QUESTIONS)

Mode	R@5	Hit@5	MRR	Pool	MH R@5	P50
Fusion, no rerank	48.2	53.1	.374	69.6	22.3	22 ms
Hybrid + rerank	66.8	72.3	.589	91.9	41.8	435 ms
held-out (7 conv.)	66.8	72.8	.586	92.4	—	—
approx. (LSH)	66.5	72.0	.587	90.6	40.9	428 ms

D. Effect of Individual Changes

Table VII lists the changes made after the baseline audit and their measured effects. Each was designed on dev data and then checked by an independent adversarial review that re-ran the numbers and probed for over-fitting; a final code review found 14 further defects, all fixed. Two findings stand out. First, enlarging the reranker pool from 20 to 100 candidates accounts for most of the LoCoMo gain: with a 20-candidate pool, gold evidence was present only 69.6% of the time. Second, the shipped approximate search was a hidden liability: with LSH on by default it dropped most candidates (benchmark Recall@5 of 57.5% versus 93.0% exact). Exact search therefore became the default, and LSH was corrected and made opt-in.

TABLE VII
MEASURED EFFECT OF EACH CHANGE

Change	Measured effect
Keep user self-disclosures (90 days)	LongMemEval retention 21.0 → 90.6%
Exact search default; corrected LSH	LSH forced on: bench. R@5 57.5 → 97.5%; LoCoMo R@5 26.8 → 66.5%
Reranker pool 20 → 100	LoCoMo R@5 55.8 → 66.8%; pool hit 69.6 → 91.9%
Keep reranked order in context	First extractive statement correct for 84.2%
Time- and link-aware reorder	Bench. R@5 93.7 → 97.5%; correct first 77.3 → 84.2%
Opt-in abstention	Unanswerable refused 0 → 17.4%; answerable kept 99.6%
Recency-mode fix	Search in recency mode no longer raises an error
Pinned evaluation clock	Tier results no longer depend on the time of day

E. Query-Type Analysis

Fig. 5 shows the effect of the structural reorder by query type. Where an answer depends on time or links, a text-

relevance cross-encoder alone ranks the correct memory first only rarely: 27.3% of temporal and 52.2% of historical questions. The structural step raises these to 59.1% and 91.3%, lifts preference questions from 82.6% to 91.3%, and raises multi-hop Recall@5 from 68.2% to 100%. Not everything improved: correction questions ("scratch that …") remain at 54.5% correct-first, and latest-update questions moved from 43.5% to 39.1%, a difference of one question.

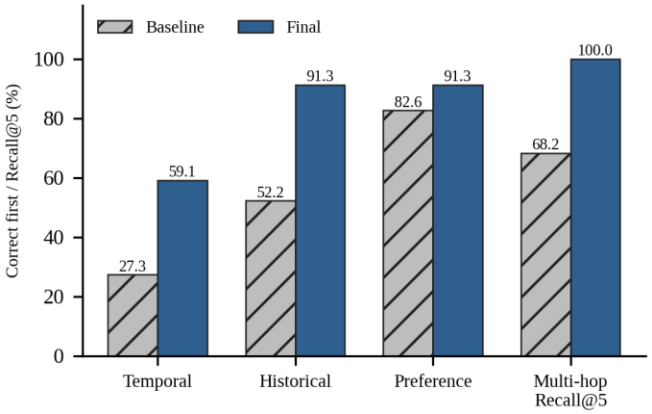


Fig. 5. Effect of the time- and link-aware structural reorder by query type (benchmark test split, %).

F. Importance Scoring, Retention, and Lifecycle

Table VIII compares the three snapshots on the measurements that depend on the importance score. The retention gain comes from the rule changes, not from the model: replacing the rules-only score by the hybrid changed no retention outcome. All 457 LongMemEval examples kept the same answer turns and the same visible share (90.6% of answer turns retrievable; 55.2% of a conversation visible), and cleanup kept five more expired memories as archive and deleted two fewer. Tier accuracy of 56.7% reflects the clock pin rather than the model (a same-day rules-only run also gives 56.7%), and "score inside the labelled range" rising to 58.3% is not better judgement, since a constant score of 0.5 lands inside 83.3% of the labelled ranges. Replay time grew from 13 s to 581 s (41×) because each memory is now embedded with BGE (about 59 ms on average). The rule-based classifier remains weak at 53.3% category accuracy (macro-F1 0.544); all 16 lifecycle, task, and supersession checks pass.

TABLE VIII
RETENTION, CLASSIFICATION, AND LIFECYCLE SNAPSHOTS

Metric	Base	Final	Current
LongMemEval answer-turn retention (%)	21.0	90.6	90.6
Mean share of conversation visible (%)	18.2	55.2	55.2
Cleanup: expired memories deleted	3,160	1,740	1,738
Cleanup: expired kept as archive	—	33	38
Category accuracy, 60 cases (%)	50.0	53.3	53.3
Tier accuracy, 60 cases (%)	55.0	53.3	56.7
Score inside labelled range (%)	48.3	50.0	58.3
Lifecycle / task checks	16/16	16/16	16/16
LongMemEval replay time	13 s	13 s	581 s

Table IX evaluates the importance model on its own synthetic test split. The ensemble reaches macro-F1 0.8912, with a 95% bootstrap interval of [0.872, 0.909] over test rows but a wider [0.809, 0.949] over whole sentences, which is the honest interval because only 86 distinct sentences were tested. This is far above a majority-class predictor (0.2529) and close to an oracle that predicts each test sentence's most common test label (0.9230), a ceiling that reads the test labels rather than a model. Table X shows where the errors are: the model never misclassifies a low-importance sentence, and its errors are all mid-importance sentences predicted as high (101 of 272, recall 62.9%) plus nine high sentences predicted as mid. On the blended probabilities before class weighting, test negative log-likelihood is 0.2193 and expected calibration error 0.0346; validation macro-F1, which was used for tuning, is 0.9141 and is therefore optimistic. Table XI lists the full configuration, including the serving check that the exported ONNX head and saved XGBoost model reproduce the training-time classes exactly (no mismatches on 792 validation and 1,008 test rows; maximum probability difference 8.4e-8 on validation and 4.0e-8 on test, and 3.2e-5 with live single-text embeddings).

TABLE IX
IMPORTANCE MODEL ON THE CSV5K TEST SPLIT (1,008 ROWS)

Model	Acc.	Bal. acc.	Macro-F1	F1 low/mid/high
Majority class	61.11	33.33	.2529	0.00 / 0.00 / 0.76
Oracle (ceiling)	91.17	92.86	.9230	1.00 / 0.84 / 0.93
Ensemble + thresholds	89.09	87.14	.8912	1.00 / 0.76 / 0.92

TABLE X
ENSEMBLE CONFUSION MATRIX (ROWS: GOLD, COLUMNS: PREDICTED)

Gold \ Pred.	Low	Mid	High	Recall (%)
Low	120	0	0	100.0
Mid	0	171	101	62.9
High	0	9	607	98.5
Precision (%)	100.0	95.0	85.7	—

TABLE XI
IMPORTANCE MODEL CONFIGURATION

Item	Value
Encoder	BAAI/bge-small-en-v1.5 (fastembed 0.8.1), frozen, 384-d
XGBoost member	525 features; depth 3; min child weight 3; 304 rounds; temperature 0.8962
Deep head	3 seeds (42/43/44): best epochs 12/7/6; temperatures 0.7544/0.9594/0.7122; averaged temperature 0.9060
Blend	0.5 XGBoost + 0.5 deep head; validation macro-F1 0.9104 / 0.9117 / 0.9100 for weights 0.25 / 0.5 / 0.75
Class weights	1 / 0.7 / 1; validation macro-F1 0.9117 → 0.9141
Class values	Low 0.3102, mid 0.5792, high 0.8372 (fit + validation rows only)
Split	Grouped by sentence, stratified 5-fold, seed 42; 286 / 71 / 86 sentences
Test calibration	NLL 0.2193; ECE 0.0346 (before class weights)

This score does not transfer to conversation. As a ranking of answer turns against the rest, the model's score has an AUC of 0.482 on user turns, where 752 of the 806 answer turns are (rules: 0.582), and 0.697 on assistant turns (rules: 0.598); the

blended score's overall AUC is 0.654 against 0.678 for the rules alone. The model labels 81.2% of turns as high importance, and 98.9% of turns have at least one numeric input more than five standard deviations from its training data, mostly because conversation turns are far longer than the at most 13-word csv5k sentences. The temperatures, blend weight, and class weights were chosen on validation data only, so only the test numbers above are unbiased.

G. Abstention and Latency

Opt-in extractive abstention (the slot rule) refuses 17.4% of unanswerable benchmark questions (up from 0%) while keeping 99.6% of answerable ones, raising balanced accuracy from 50.0% to 58.5%. We also compared a QNLI-style answerability model [18] (Table XII). It refuses far more (52.2% on the benchmark) but answers only 97.2%, 78.5%, and 76.1% of answerable questions on the benchmark, LongMemEval, and LoCoMo. The adoption rule required at least 95% (benchmark) or 90% (external) of answerable questions still answered and a balanced-accuracy gain of at least three points on both external sets; the slot rule fails the gain test and the QNLI model fails the answer-rate test, so abstention stays opt-in and the QNLI model was not shipped. Median search latency increased about 3–4× (benchmark 85 → 246 ms, LoCoMo 119 → 435 ms), mostly because of the larger reranker pool; setting the pool to 50 keeps most of the LoCoMo gain at about half the reranker cost.

TABLE XII
ABSTENTION DESIGNS: UNANSWERABLE REFUSED / BALANCED ACCURACY (%)

Design	Bench.	LongMem.	LoCoMo
None (default)	0.0 / 50.0	0.0 / 50.0	0.0 / 50.0
Slot rule (opt-in)	17.4 / 58.5	6.7 / 52.5	1.6 / 50.0
QNLI model (not shipped)	52.2 / 74.7	33.3 / 55.9	36.3 / 56.2

VI. DISCUSSION

A. Answering the Research Question

Relational and temporal information helps most where similarity is least informative. In the ablation, adding the graph to vector search raised benchmark Recall@5 from 89.7% to 94.1%, and the time- and link-aware reorder lifted correct-first accuracy by 31.8 points on temporal and 39.1 points on historical questions and multi-hop Recall@5 to 100%. On raw dialogue (LoCoMo), the graph channel contributes little because no relations are stored, and most of the gain comes from the larger pool and the cross-encoder. Graph retrieval is thus complementary rather than universal: its value scales with the explicit relational and temporal structure available at write time. We note that the arm differences are single-run point estimates, and we report paired intervals only for the baseline-versus-final comparisons.

B. Measurement Audit

An earlier internal measurement reported considerably higher figures that did not survive re-measurement. The benchmark questions had been templated alongside the rules (procedural questions scored 5% because ingestion merged 19 of their 20 memories); the extractive answerer in fact refused none of the unanswerable questions, and the abstention figure had counted answerable questions as correct; the shipped approximate-search path had never been measured; and the classification cases had been used to develop the rules. We rebuilt the harness, benchmark, and caches and report only rebuilt numbers. We include this audit because inflated memory benchmarks are easy to produce and because it changed system defaults.

C. A Negative Result: The Trained Importance Model

An earlier version of CogniMem offered optional XGBoost and neural scorers trained on the Personal Facts collection [19] to predict fact validity and duration (test macro-F1 0.6114 for XGBoost and 0.6732 for the deeper neural head). They were never on by default and predicted a different target, so they were replaced by the csv5k importance ensemble as 25% of every score. The results above show that this change was neutral at best: the model is accurate on its own synthetic data (macro-F1 0.891) but, at a fixed 25% weight, it did not change the visibility of a single answer turn, it does not recognise answer-bearing turns (AUC 0.482 on user turns), and it made replay 41 times slower. We report this rather than omit it because it shows how a model that looks strong on in-distribution synthetic data can be inert, or costly, in the deployment domain. Human-rated, in-domain importance labels are needed before the model deserves more weight.

D. Relation to the Base Review

The proof of concept in [1] compared a hybrid design (relevance 0.704) against its best single component (0.626) using keyword-overlap relevance on 20 queries over 12 documents. Our protocol differs in scale, retrieval models, and metrics, so the figures are not comparable; the contribution here is a complete, reproducible implementation of the recommended direction, measured on larger and external data. Of the memory types discussed in [1], CogniMem covers semantic, episodic, procedural, preference, and task memory; emotional memory and a finite-state-machine layer for procedural flows are not part of the current system.

E. Limitations and Threats to Validity

The benchmark and classification cases are AI-authored and unreviewed, and the dev and test personas share templates; LoCoMo is generated dialogue, and its first three conversations informed the pool size. csv5k is synthetic (AI-written sentences and scores, not human ratings) and two lifecycle-check texts appear in it verbatim; the 25% weight is a fixed choice, not a tuned one, and records stored before the switch keep their rule-based score. Answer metrics use the extractive generator only; live Gemini correctness, citation usefulness, and conflict decisions need independent human review, and exact-quote validation cannot prove semantic truth. No competing memory system was run under the same protocol. Correction and latest-update queries are still often answered with an older statement, the classifier reaches only 53% on its test cases, and the larger pool costs 3–4× latency, measured on one machine. Local JSONL and SQLite storage suits research and small

deployments, not production scale; one lifecycle check and one task test fail rarely because of a wall-clock race in task updates.

VII. Conclusion and Future Work

We presented CogniMem, a local-first hybrid memory architecture that implements the recommendation of a recent literature review: it classifies, scores, tiers, reconciles, and persists conversational memories; keeps history in a bitemporal graph; and answers from fused, re-ranked, time-aware evidence with citations. In a rebuilt and audited evaluation, the graph raised benchmark Recall@5 from 89.7% to 94.1% over vector search alone, and the shipped configuration reached 97.5% Recall@5 on the controlled benchmark, 66.8% on LoCoMo, and 90.6% fact retention, with the largest gains on temporal, historical, and multi-hop queries. The trained importance model, by contrast, proved inert on real conversation.

Future work includes (i) a human-validated, conversation-disjoint benchmark and independent ratings of live LLM answers; (ii) human-rated, in-domain importance labels to retrain or re-weight the scorer; (iii) repairing correction and latest-update queries, likely through claim-level supersession tracking; (iv) head-to-head comparisons with MemGPT, Mem0, and LangMem under the same protocol; (v) feedback-driven confidence updates from retrieval success and user corrections; (vi) emotional memory and procedural state machines as described in [1]; and (vii) production-grade storage, access control, and consent management. Code, benchmark seeds, model files, and evaluation reports are available in the public repository [20].

References

[1] N. A. Akbar, R. Dembani, B. Lenzitti, and D. Tegolo, "RAG-driven memory architectures in conversational LLMs—A literature review with insights into emerging agriculture data sharing," *IEEE Access*, vol. 13, pp. 123855–123880, 2025, doi: 10.1109/ACCESS.2025.3589241.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023, *arXiv:2310.08560*.

[4] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," 2025, *arXiv:2504.19413*.

[5] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, 2024.

[6] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.

[7] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," 2023, *arXiv:2309.07597*.

[8] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2009, pp. 758–759.

[9] R. Nogueira and K. Cho, "Passage re-ranking with BERT," 2019, *arXiv:1901.04085*.

[10] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[11] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 494–514, Feb. 2022.

[12] R. Snodgrass and I. Ahn, "A taxonomy of time in databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1985, pp. 236–246.

[13] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[14] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," 2024, *arXiv:2410.10813*.

[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.

[16] G. Comanici *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," 2025, *arXiv:2507.06261*.

[17] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[18] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[19] "Personal Facts (MSC)," Hugging Face dataset *adugeen/personal-facts-msc*. [Online]. Available: https://huggingface.co/datasets/adugeen/personal-facts-msc

[20] J. Remingston L., S. Suhas, K. Shreenagesh, and T. Madanwad, "CogniMem: Local-first temporal memory for personalized AI assistants," 2026. [Online]. Available: https://github.com/JosephRemingston/capstone